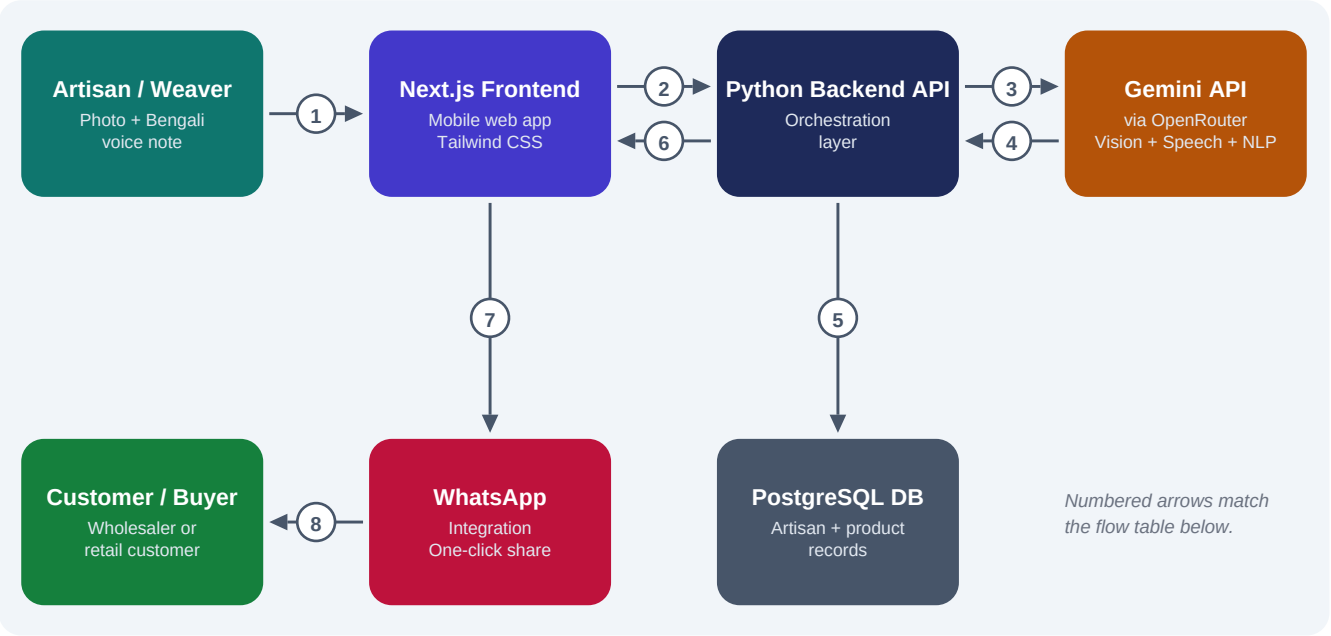
Vernacular AI E-Commerce Enabler

Technical Architecture & Overview

Participant: Tanmay Dey | **Institution:** Brainware University (B.Tech CSE-AIML)
Category: Employment, Livelihood & MSMEs (Junior Category)

1. System Architecture Diagram



1	Artisan uploads a product photo and a Bengali voice note to the Next.js frontend.
2	Frontend sends the media to the Python backend via REST API.
3	Backend routes the image and audio to the Gemini API.
4	AI returns the parsed SEO description, tags and pricing to the backend.
5	Backend stores the product record in PostgreSQL.
6	Backend returns the formatted digital card to the frontend.
7	Artisan exports the card with one click to WhatsApp.
8	WhatsApp shares the catalog link with the customer / buyer.

2. Technical Stack Breakdown

The platform is designed as a lightweight, mobile-first web application, intentionally avoiding the friction of native app store downloads for rural users.

- **Frontend Interface:** Built with Next.js and Tailwind CSS. This ensures a highly responsive, mobile-optimized user experience that loads efficiently even on low-bandwidth networks (3G/4G).
- **Backend Logic:** Powered by Python. The backend serves as the orchestration layer, handling asynchronous media uploads, managing API requests, and executing business logic securely.
- **Database:** PostgreSQL is utilized for robust relational data management, mapping individual artisans to their specific product inventories, categories, and localized metadata.

  AI & Machine Learning Layer: The core intelligence relies on multimodal AI processing routed through OpenRouter, specifically utilizing Google Gemini models. This handles:

- **Vision Processing:** Analyzing the uploaded handloom image to identify materials, colors, and weave patterns (e.g., identifying a Jamdani motif).
- **Speech-to-Text & NLP:** Transcribing the artisan's Bengali voice note and translating it into a structured, SEO-friendly English/Bengali product description.

3. Step-by-Step Data Flow

1	**Ingestion:** An artisan opens the web app on their smartphone, captures a photo of a newly woven saree, and taps the microphone to record a brief description in Bengali (e.g., "This is a red and white Phulia cotton Taant, price 850 rupees").
2	**Processing:** The Next.js frontend packages this media and securely transmits it to the Python backend.
3	**AI Generation:** The backend dispatches the audio and image to the Gemini API. The AI extracts the visual features and the spoken details, merging them to generate a professional e-commerce listing (Title, Bullet Points, SEO Tags).
4	**Storage & Retrieval:** The generated data is saved to the PostgreSQL database, creating a persistent digital inventory for that specific artisan.
5	**Distribution:** The frontend displays the polished product card. The artisan can tap a "Share to WhatsApp" button, which generates a clean, formatted text block and URL to share directly with wholesalers or retail customers.

4. Security and Scalability

- **Stateless Processing:** By leveraging cloud-based LLMs through OpenRouter, the application remains lightweight. Heavy computational tasks (vision and language processing) are offloaded, allowing the platform to run on cost-effective hosting.
- **Scalable Architecture:** The separation of the Next.js frontend and Python backend allows each service to scale independently based on user traffic.
- **Modular Language Expansion:** Expanding the tool from West Bengal to Bihar or Jharkhand requires zero changes to the underlying infrastructure; it simply involves adjusting the AI prompt instructions to accept Hindi or Bhojpuri audio inputs.